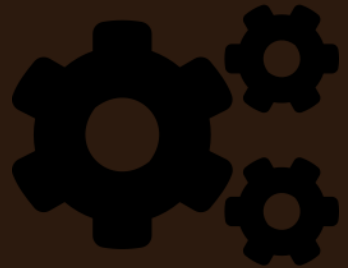


CHAPTER 9

Software Configuration Management

Presented By
Rifa Zumana
Lecturer
Dept. of CSE, NBIU



What is SCM?

Software Configuration Management: a quality assurance activity throughout the software process

SCM is a set of activities developed to manage change throughout the entire software lifecycle.



Identify Change

Recognize and catalog what has changed in any software artifact.



Control Change

Gate every modification through a formal approval workflow.



Ensure Proper Implementation

Verify that approved changes have been correctly applied.



Report Changes

Communicate modifications to all interested stakeholders.

Why Do We Need SCM?

The need for coordination and streamlined processes

01

Multiple Contributors

Large teams work on the same project simultaneously, editing shared files and components — without coordination, chaos ensues.

02

Multiple Document Versions

Requirements, designs, and code evolve continuously. Tracking which version is current (and valid) is a serious management challenge.

03

Multiple Customer Releases

Different customers may run different release versions. SCM ensures each release can be reproduced, patched, and traced.



What Are These Changes?

Changes originate from multiple sources and ripple across all artifacts

Change Sources

Business Requirements

Technical Requirements

User Requirements

Software
Models

Affected Artifacts

Code

Test Plans

Project Plan

Other Docs

Changes cascade from requirements → models → every project artifact

Software Configuration Items (SCI)

Named elements of information created during the software engineering process

The entire collection of information produced as part of the software process is called the software configuration. Each individually named element within that configuration is an SCI.

Computer Programs

- Source code
- Object code
- Executables

Documents

- Requirements
- Design specs
- User manuals

Data / Contents

- Test data
- Config files
- Databases

Output of the Software Process

Baselines

Formally reviewed and approved milestones in software development

IEEE Std. 610.12-1990: A baseline is a specification or product formally reviewed and agreed upon, serving as a basis for further development that can only be changed via formal change control procedures.

Engineering
Tasks

SCIs
(Draft)

Technical
Review

Approved
SCIs

Project
Database

Baseline Types

System Specification

Software Requirements

Design Specification

Source Code

Test Plans / Procedures / Data

Operational System

The SCM Repository

A managed store for all software configuration data — much more than a simple database



Data Integrity

Prevents corruption and unauthorized changes to stored configuration items.



Information Sharing

Provides controlled access so all team members can retrieve what they need.



Tool Integration

Integrates with compilers, editors, testing tools, and issue trackers.



Data Integration

Links related objects across the repository (e.g., requirements to code).



Methodology Enforcement

Enforces the team's chosen software process and workflow rules.



Document Standardization

Ensures consistent format and naming conventions across all documents.

Repository Content Areas

The SCM repository contains six interconnected content domains

Business Content

- Business rules
- Business functions
- Organization structure
- Information architecture

Model Content

- Use cases, Analysis model
- Scenario/Flow/Class diagrams
- Design model, Tech metrics

Construction Content

- Source code
- Object code
- System build instructions

V&V Content

- Test cases & scripts
- Test results
- Quality metrics

Project Mgmt. Content

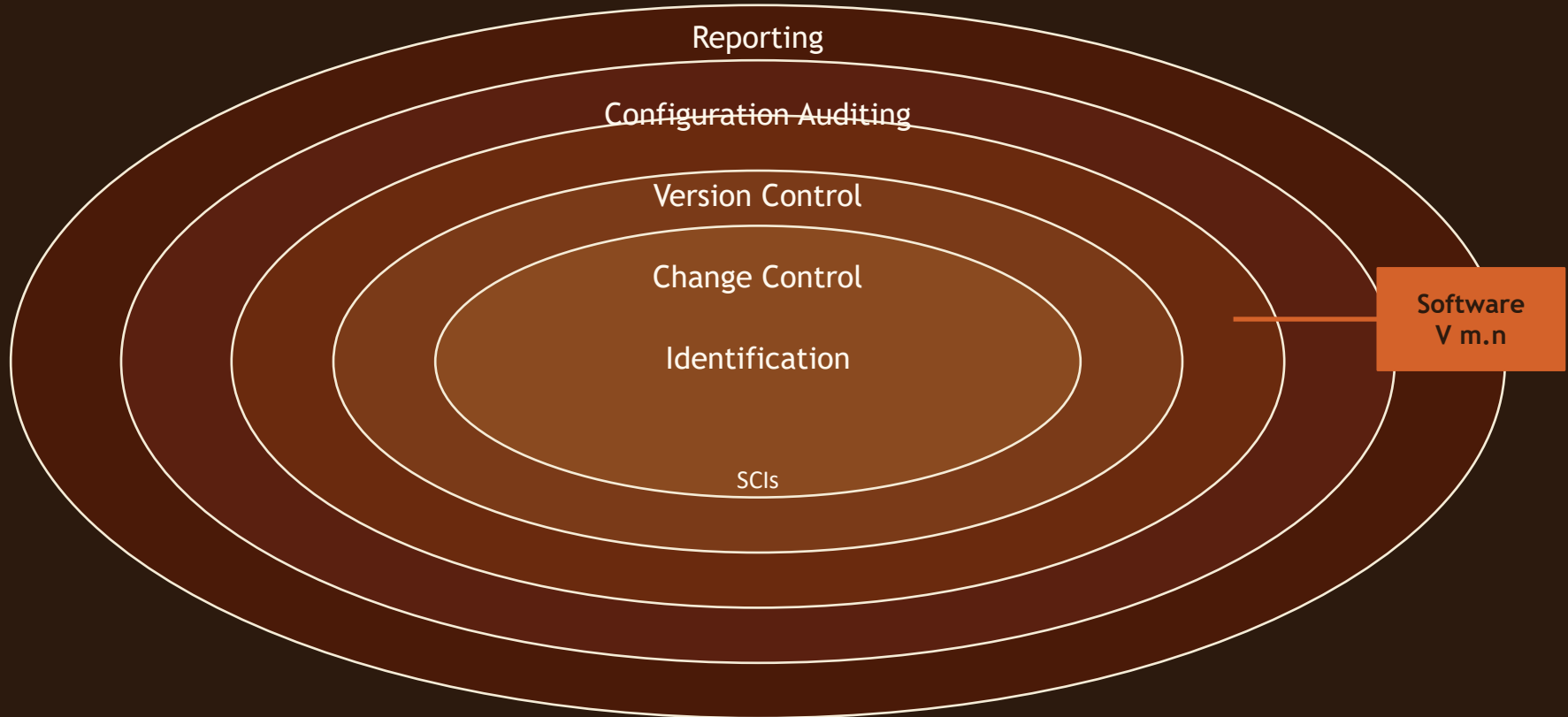
- Estimates, Schedule
- SCM/SQA requirements
- Change requests & reports

Documents

- Project plan, System spec
- Requirements & Design docs
- Test plans, User manual

Layers of the SCM Process

SCIs flow outward through layers, ultimately forming versioned software releases



Each layer adds process controls. SCIs begin at the core (identification) and flow outward to become versioned software releases.

Version Control

Combines procedures and tools to manage different versions of configuration objects

A version control system is directly integrated with four major capabilities that together manage the evolution of software:

1

Project Database (Repository)

Stores all relevant configuration objects and their history in a structured, queryable store.

2

Version Management

Stores all versions of a configuration object — or reconstructs any version from incremental differences.

3

Make Facility

Collects all relevant SCIs and constructs a specific, reproducible version of the entire software system.

4

Issue / Bug Tracking

Records and tracks the status of all outstanding issues associated with each configuration object.

Configuration Audit

Ensuring changes have been properly implemented — complementing technical reviews

A technical review checks the technical correctness of a modified object. A configuration audit goes further by verifying the process was followed correctly.

1

Was the change in the ECO actually made?

No unauthorized changes should have been added.

2

Was the correct software process followed?

Standards must be properly applied throughout.

3

Has the change been highlighted in the SCI?

Change date and author must be recorded.

4

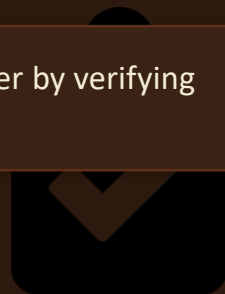
Were SCM procedures for recording followed?

Reporting procedures must be fully observed.

5

Have all related SCIs been properly updated?

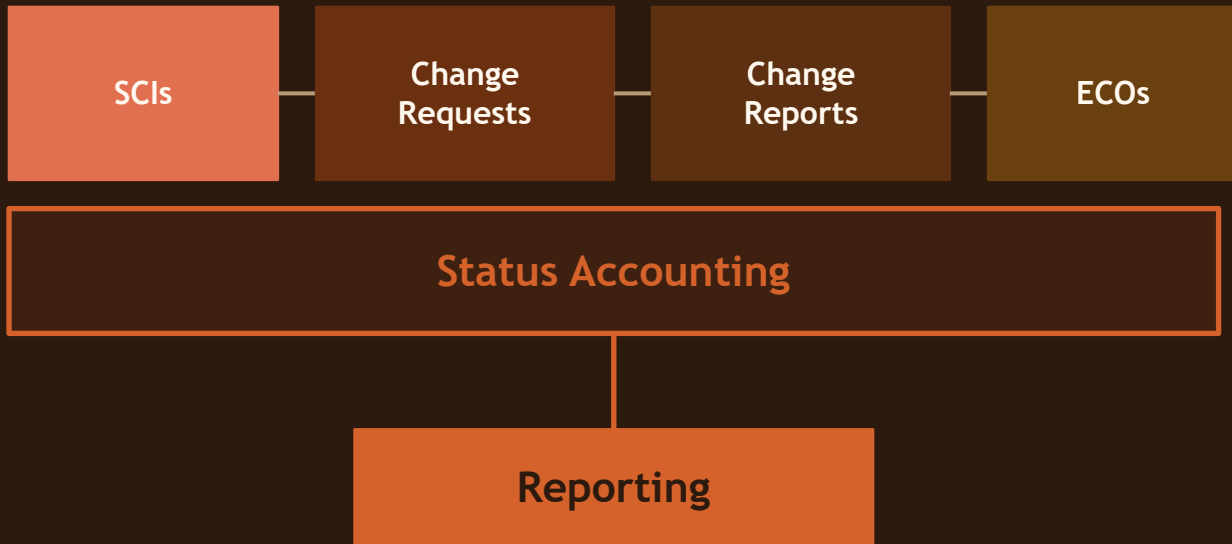
Dependent objects must reflect the change.



Configuration Status Reporting

Also called status accounting — the communication layer of SCM

Status reporting answers: (1) What happened? (2) Who did it? (3) When did it happen? (4) What else will be affected?



Report Content

- Change ID & description
- Date & author
- Baseline affected
- Objects modified
- Status & outcome

Configuration Management Tools

Modern tooling that implements SCM principles in real-world environments



Git

Version Control

The world's most widely used distributed version control system. Tracks every change to source code with full history.

Terraform

Infrastructure

Infrastructure-as-code tool. Manages and versions cloud infrastructure configuration across providers.

Ansible

Automation

Agentless automation platform for configuration management, application deployment, and orchestration.

Chef

Config Mgmt.

Treats infrastructure as code. Defines configurations as 'recipes' that keep systems in a desired state.

Puppet

Config Mgmt.

Declarative language to describe system configuration. Enforces and remediates drift automatically.

Docker

Containerization

Packages applications with all their dependencies into containers: ensuring consistent environments.

CONFIGURATION MANAGEMENT TOOLS



Terraform

'SALTSTACK



puppet



git



CHEF™



ANSIBLE



docker

Thank You 😊